МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Факультет безопасности информационных технологий

Направление подготовки: 10.03.01 Информационная безопасность

Образовательная программа: «Технологии защиты информации»

Дисциплина:

«Информационная безопасность баз данных»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №5

«Резервирование БД и восстановление по контрольным точкам»

Выполнил студент(ы):

группа.поток ИББД 1.7

Пахомов Владимир Юрьевич / 

ФИО *Подпись*

Сивков Иван Александрович / 

ФИО *Подпись*

Булин Михаил Михайлович / 

ФИО *Подпись*

Проверил:

Ярцева Наталия Андреевна / _____

ФИО *Подпись*

_____

Отметка о выполнении (один из вариантов:

отлично, хорошо, удовлетворительно, зачтено)

_____

Дата

Санкт-Петербург

2026

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **3**

1 Создание резервной копии БД согласно расписанию **4**

2 Фиксация контрольной точки и внесение изменений **5**

2.1 Создание базового снимка и контрольной точки 5

2.2 Внесение деструктивных изменений 5

3 Откат к контрольной точке (Point-In-Time Recovery) **6**

3.1 Процедура восстановления 6

3.2 Проверка целостности восстановленных данных 6

4 Анализ откаченных изменений **7**

4.1 Анализ с помощью системы логирования СУБД (ЛР3) 7

4.2 Анализ с помощью средств системы резервирования (pg_waldump) 7

ЗАКЛЮЧЕНИЕ **8**

ВВЕДЕНИЕ

Цель работы: получение практических навыков разработки политик и регламентов резервного копирования баз данных, настройки механизмов непрерывной архивации журналов упреждающей записи (Write-Ahead Logging, WAL), моделирования инцидентов деструктивной модификации данных, проведения аварийного восстановления на определенный момент времени (Point-In-Time Recovery, PITR) до контрольной точки, а также последующего криминалистического анализа аннулированных изменений.

Обеспечение доступности и целостности информации выступает ключевой задачей информационной безопасности в соответствии с базовой триадой CIA (Confidentiality, Integrity, Availability). В автоматизированной информационной системе морских контейнерных перевозок непреднамеренные программные сбои или несанкционированные воздействия на таблицы рейсов, заказов и контейнеров способны парализовать логистические процессы морского терминала.

В рамках лабораторной работы на базе созданной в ЛР1–4 СУБД PostgreSQL (`container_cargo_db`) решаются четыре основные задачи:

1. организация регулярного создания резервных копий по расписанию с непрерывной архивацией журналов WAL;
2. установка именованной контрольной точки восстановления (*Restore Point*) и внесение случайных деструктивных изменений в таблицы БД;
3. проведение процедуры отката состояния СУБД к созданной контрольной точке (PITR);
4. анализ возможности просмотра и аудита откаченных изменений с помощью системы логирования СУБД (`main_log` из ЛР3) и средств системы резервирования (`pg_waldump`).

1 СОЗДАНИЕ РЕЗЕРВНОЙ КОПИИ БД СОГЛАСНО РАСПИСАНИЮ

В соответствии с заданием организуем регулярное резервное копирование базы данных по расписанию с использованием утилиты `pg_basebackup` и механизмов непрерывной архивации журналов предзаписи (Write-Ahead Logging, WAL).

Для включения архивации транзакционных журналов в конфигурацию СУБД внесены следующие параметры:

```
ALTER SYSTEM SET wal_level = 'replica';
ALTER SYSTEM SET archive_mode = 'on';
ALTER SYSTEM SET archive_command = 'test ! -f /var/backups/postgres/wal_archive/%f && cp %p /var/
    backups/postgres/wal_archive/%f';
ALTER SYSTEM SET archive_timeout = 900;
```

Автоматическое создание физических снимков настроено с помощью системного планировщика `cron` для пользователя `postgres` (запуск ежедневно в 02:00 ночи):

```
# Расписание в crontab
0 2 * * * pg_basebackup -h localhost -U postgres -D /var/backups/postgres/base_backups/base_backup -
    Ft -z -X fetch
```

Подтверждение настройки расписания в планировщике и параметров архивации приведено на рисунке 1.

```
ivan@ivan-bogatyr:~/Загрузки$ sudo -u postgres crontab -l
sudo -u postgres psql -c "SHOW archive_mode; SHOW archive_command;"
0 2 * * * pg_basebackup -h localhost -U postgres -D /var/backups/postgres/base_backups/base_$(date +\%Y\%m\%d) -Ft -z -X fetch
 archive_mode
--------------
 on
(1 row)

                                  archive_command
-------------------------------------------------------------------------------------------
 test ! -f /var/backups/postgres/wal_archive/%f && cp %p /var/backups/postgres/wal_archive/%f
(1 row)
```

Рисунок 1 — Настройка планировщика cron и параметров архивации WAL

2 ФИКСАЦИЯ КОНТРОЛЬНОЙ ТОЧКИ И ВНЕСЕНИЕ ИЗМЕНЕНИЙ

2.1 Создание базового снимка и контрольной точки

Перед проведением эксперимента сформирован актуальный базовый физический снимок каталога данных:

```
sudo -u postgres pg_basebackup -h localhost -D /var/backups/postgres/base_backups/base_snapshot -Fp
    -Xs -P
```

Затем администратор СУБД фиксирует эталонное состояние данных и создает именованную контрольную точку восстановления (*Restore Point*):

```
SELECT id, cargo_name, total_price, cargo_weight FROM orders LIMIT 3;
SELECT pg_create_restore_point('rp_cargo_safe_state');
```

Результат фиксации исходных цен и создания контрольной точки представлен на рисунке 2.

```
container_cargo_db=# SELECT id, cargo_name, total_price, cargo_weight FROM orders LIMIT 3;
 id |          cargo_name          | total_price | cargo_weight
----+------------------------------+-------------+--------------
  1 | Электроника и микрочипы      |   450000.00 |     18500.00
  2 | Текстильная продукция        |   210000.00 |      9200.00
  3 | Автозапчасти и комплектующие |   520000.00 |     24000.00
(3 rows)

container_cargo_db=# SELECT pg_create_restore_point('rp_cargo_safe_state');
 pg_create_restore_point
-------------------------
 0/50000C8
(1 row)
```

Рисунок 2 — Фиксация исходных цен и создание контрольной точки rp_cargo_safe_state

2.2 Внесение деструктивных изменений

Смоделируем компрометацию учетной записи: обнуление цен заказов, удаление контейнеров и внедрение фиктивного контрагента:

```
UPDATE orders SET total_price = 0.00, cargo_weight = 1.00 WHERE id IN (1, 2, 3);
DELETE FROM order_containers WHERE id IN (1, 2, 3);
INSERT INTO clients (name, email, phone, address)
VALUES ('Phantom Logistics Ltd.', 'hacker@darknet.io', '+000-000-0000', 'Unknown Island 99');
SELECT pg_switch_wal();
```

Фиксация поврежденных данных (обнуленные цены и искаженный вес) и автоматическое срабатывание триггера аудита `main_log` (разработанного в ЛР3) показаны на рисунке 3.

```
container_cargo_db=#
UPDATE orders SET total_price = 0.00, cargo_weight = 1.00 WHERE id IN (1, 2, 3);
DELETE FROM order_containers WHERE id IN (1, 2, 3);
INSERT INTO clients (name, email, phone, address)
VALUES ('Phantom Logistics Ltd.', 'hacker@darknet.io', '+000-000-0000', 'Unknown Island 99');
SELECT pg_switch_wal();
UPDATE 3
DELETE 3
INSERT 0 1
 pg_switch_wal
---------------
 0/500DC90
(1 row)

container_cargo_db=#
SELECT id, cargo_name, total_price, cargo_weight FROM orders WHERE id IN (1, 2, 3);
SELECT log_item_id, operation_type, table_name, user_operator FROM main_log ORDER BY log_item_id DESC LIMIT 4;
 id |          cargo_name          | total_price | cargo_weight
----+------------------------------+-------------+--------------
  1 | Электроника и микрочипы      |        0.00 |         1.00
  2 | Текстильная продукция        |        0.00 |         1.00
  3 | Автозапчасти и комплектующие |        0.00 |         1.00
(3 rows)

 log_item_id | operation_type |    table_name    | user_operator
-------------+----------------+------------------+---------------
          10 | I              | clients          | postgres
           9 | D              | order_containers | postgres
           8 | D              | order_containers | postgres
           7 | D              | order_containers | postgres
(4 rows)
```

Рисунок 3 — Отражение повреждения данных (цены 0.00) и аудит операций в main_log

3 ОТКАТ К КОНТРОЛЬНОЙ ТОЧКЕ (POINT-IN-TIME RECOVERY)

3.1 Процедура восстановления

Откат состояния СУБД производится через процедуру аварийного восстановления на определенный момент времени:

```
sudo systemctl stop postgresql
sudo mv /var/lib/postgresql/16/main /var/lib/postgresql/16/main_corrupted
sudo cp -r /var/backups/postgres/base_backups/base_snapshot /var/lib/postgresql/16/main
sudo chown -R postgres:postgres /var/lib/postgresql/16/main

sudo -u postgres touch /var/lib/postgresql/16/main/recovery.signal
sudo -u postgres bash -c 'cat >> /var/lib/postgresql/16/main/postgresql.auto.conf' << 'EOF'
restore_command = 'cp /var/backups/postgres/wal_archive/%f %p'
recovery_target_name = 'rp_cargo_safe_state'
recovery_target_action = 'promote'
EOF

sudo systemctl start postgresql
```

3.2 Проверка целостности восстановленных данных

После запуска СУБД накатывает сегменты WAL из архива ровно до созданной контрольной точки. Выполним проверку состояния таблиц:

```
SELECT id, cargo_name, total_price, cargo_weight FROM orders WHERE id IN (1, 2, 3);
SELECT id, order_id, container_id, seal_number FROM order_containers WHERE id IN (1, 2, 3);
SELECT * FROM clients WHERE email = 'hacker@darknet.io';
```

Результат проверки представлен на рисунке 4. Все цены и контейнеры вернулись в исходное состояние, фиктивный контрагент удален.

```
container_cargo_db=#
SELECT id, cargo_name, total_price, cargo_weight FROM orders WHERE id IN (1, 2, 3);
SELECT id, order_id, container_id, seal_number FROM order_containers WHERE id IN (1, 2, 3);
SELECT * FROM clients WHERE email = 'hacker@darknet.io';
 id |          cargo_name          | total_price | cargo_weight
----+------------------------------+-------------+--------------
  1 | Электроника и микрочипы      |   450000.00 |     18500.00
  2 | Текстильная продукция        |   210000.00 |      9200.00
  3 | Автозапчасти и комплектующие |   520000.00 |     24000.00
(3 rows)

 id | order_id | container_id | seal_number
----+----------+--------------+-------------
  1 |        1 |            1 | SL-998811
  2 |        2 |            3 | SL-445522
  3 |        3 |            2 | SL-776633
(3 rows)

 id | name | email | phone | address
----+------+-------+-------+---------
(0 rows)
```

Рисунок 4 — Успешное восстановление данных к контрольной точке rp_cargo_safe_state

4 АНАЛИЗ ОТКАЧЕННЫХ ИЗМЕНЕНИЙ

4.1 Анализ с помощью системы логирования СУБД (ЛР3)

При откате базы данных таблица `public.main_log` также откатывается в состояние до инцидента. Следовательно, при классическом PITR внутренний журнал аудита не содержит записей о совершенных деструктивных действиях. Для расследования инцидентов журнал должен выгружаться из сохраненной копии `main_corrupted` до ее перезаписи.

4.2 Анализ с помощью средств системы резервирования (pg_waldump)

Файлы архивных журналов в директории `wal_archive` сохраняются независимо от процедур отката базы. С помощью утилиты `pg_waldump` выполнено декодирование архивного WAL-файла, содержащего события после контрольной точки:

```
sudo pg_waldump /var/backups/postgres/wal_archive/000000010000000000000005 | grep -E "RESTORE_POINT|
    Heap"
```

Вывод утилиты приведен на рисунке 5.

```
--- Файл: 000000010000000000000005 ---
rmgr: XLOG        len (rec/tot):     98/    98, tx:          0, lsn: 0/05000060, prev 0/05000028, desc: RESTORE_POINT rp_cargo_safe_state
rmgr: Heap        len (rec/tot):     65/  1069, tx:        951, lsn: 0/050001E8, prev 0/050001B0, desc: HOT_UPDATE old_xmax: 951, old_off: 1, old_infobits: [], flags: 0x00, new_xmax: 0, new_off: 9, blkref #0: rel
1663/16836/16880 blk 0 FPW
rmgr: Heap        len (rec/tot):     83/    83, tx:        951, lsn: 0/05000618, prev 0/050001E8, desc: HOT_UPDATE old_xmax: 951, old_off: 2, old_infobits: [], flags: 0x60, new_xmax: 0, new_off: 10, blkref #0: rel
 1663/16836/16880 blk 0
rmgr: Heap        len (rec/tot):     86/    86, tx:        951, lsn: 0/05000670, prev 0/05000618, desc: HOT_UPDATE old_xmax: 951, old_off: 3, old_infobits: [], flags: 0x60, new_xmax: 0, new_off: 11, blkref #0: rel
 1663/16836/16880 blk 0
rmgr: Heap2       len (rec/tot):     60/  8080, tx:        951, lsn: 0/050006C8, prev 0/05000670, desc: PRUNE snapshotConflictHorizon: 0, nredirected: 1, ndead: 0, blkref #0: rel 1663/16836/1249 blk 61 FPW
rmgr: Heap        len (rec/tot):    591/   591, tx:        951, lsn: 0/050026D8, prev 0/05002670, desc: INSERT+INIT off: 1, flags: 0x00, blkref #0: rel 1663/16836/17015 blk 0
rmgr: Heap        len (rec/tot):    587/   587, tx:        951, lsn: 0/050029C8, prev 0/05002988, desc: INSERT off: 2, flags: 0x00, blkref #0: rel 1663/16836/17015 blk 0
rmgr: Heap        len (rec/tot):    607/   607, tx:        951, lsn: 0/05002C58, prev 0/05002C18, desc: INSERT off: 3, flags: 0x00, blkref #0: rel 1663/16836/17015 blk 0
rmgr: Heap        len (rec/tot):     59/   563, tx:        952, lsn: 0/05002F20, prev 0/05002EF8, desc: DELETE xmax: 952, off: 1, infobits: [KEYS_UPDATED], flags: 0x00, blkref #0: rel 1663/16836/16915 blk 0 FPW
rmgr: Heap        len (rec/tot):     54/    54, tx:        952, lsn: 0/05003158, prev 0/05002F20, desc: DELETE xmax: 952, off: 2, infobits: [KEYS_UPDATED], flags: 0x00, blkref #0: rel 1663/16836/16915 blk 0
```

Рисунок 5 — Декодирование журнала WAL утилитой pg_waldump (контрольная точка и откаченные операции)

В журнале четко идентифицируются:

- маркер контрольной точки `RESTORE_POINT rp_cargo_safe_state`;
- транзакции `HOT_UPDATE` по обнулению стоимостей заказов;
- операции `INSERT` и `DELETE` по удалению связей контейнеров.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были освоены и практически применены технологии резервирования, обеспечения высокой доступности и аварийного восстановления реляционной базы данных «Информационная система морских контейнерных перевозок» в СУБД PostgreSQL.

В рамках работы достигнуты следующие результаты:

1. Разработан регламент резервного копирования данных, включающий создание физических снимков (`pg_baseba`
   и непрерывную архивацию журналов предзаписи (`WAL-archiving`); процесс автоматизирован посредством планировщика `cron`.
2. Введена в эксплуатацию именованная контрольная точка восстановления `rp_cargo_safe_state`, зафиксировавшая эталонное состояние таблиц базы данных.
3. Смоделирован сценарий деструктивного нарушения целостности данных (обнуление цен, удаление связей контейнеров, внедрение фиктивных записей контрагентов).
4. Успешно выполнен процесс отката базы данных к контрольной точке по технологии Point-In-Time Recovery (PITR); верифицировано восстановление целостности данных.
5. Проведен комплексный анализ аннулированных транзакций: раскрыта специфика поведения встроенной таблицы-лога (`main_log`) при откате кластера и продемонстрирован независимый метод форензик-анализа через декодирование архивных сегментов WAL утилитой `pg_waldump`.